

Внутренняя кухня: как связаны HTML, CSS, JS и C#

Не презентация для пользователей, а визуальная карта проекта для быстрой ориентации в коде: куда приходит событие, кто вызывает backend и где искать нужную логику.

KanbanBoard_HTML.html

kanban.js

kanban.css

SOYUZ_UPLOAD_*.cs

```
onclick → window.kbDrop()
JS → InvokeTemplate()
C# → case "MoveTask"
PLM → KanbanTask
```

HTML

DOM, Liquid, обработчики событий

JS

состояние, UX, bridge-вызовы

CSS

layout, модалки, статусы

C#

данные, права, операции PLM

Проект держится на шести runtime-файлах: один рисует экран, один рулит UX, один красит, два выполняют C#-логику, один открывает доску.

backend

SOYUZ_UPLOAD_KanbanScreen_script.cs

Главный диспетчер: ``Invoke``, ``BeforeRender``, CRUD, роли, отчёты, вложения, чат.

task hook

SOYUZ_UPLOAD_KanbanTask_script.cs

``OnBeforeSave``, уведомления при назначении, сервисная логика шаблона.

markup

KanbanBoard_HTML.html

Liquid-шаблон, DOM-узлы, ``onclick``, ``ondrop``, ``id``, ``class``.

client

kanban.js

IIFE, ``window.*`` API, состояние UI, вызовы ``InvokeTemplate``.

styles

kanban.css

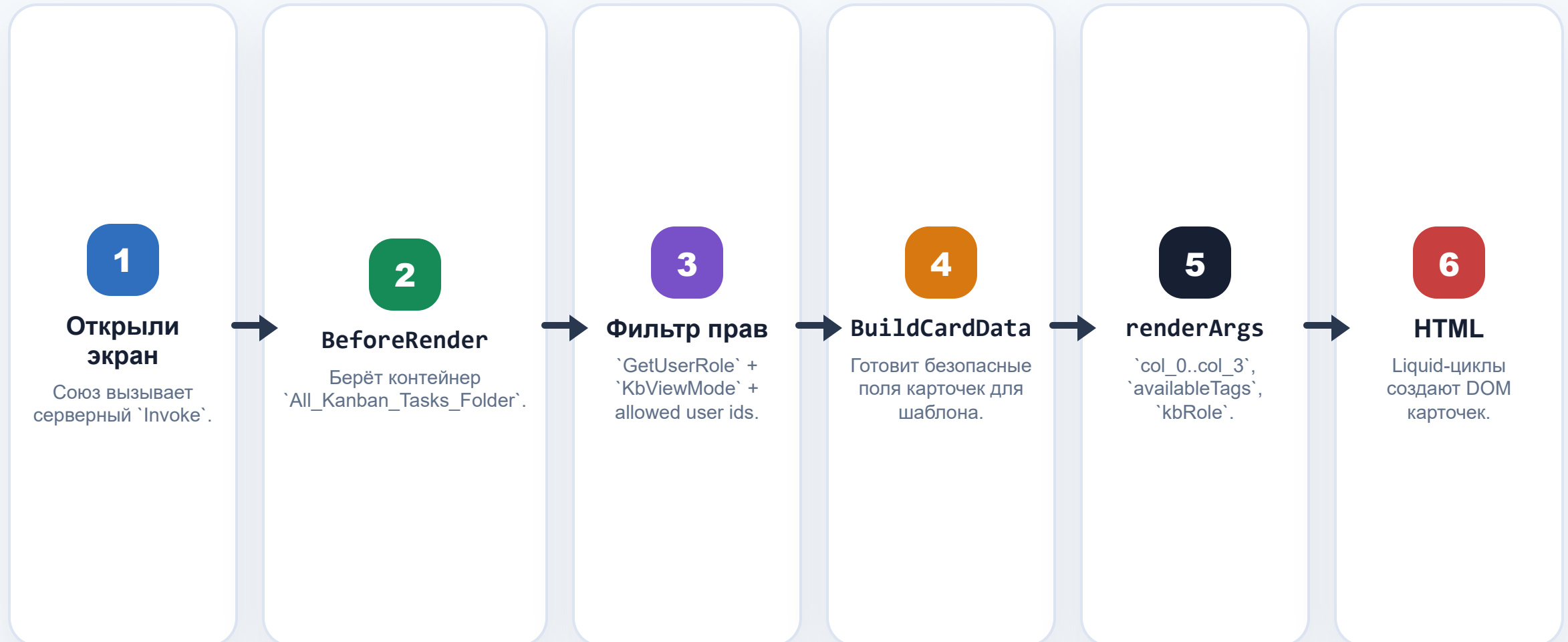
``..kb-*``, ``..tcm-*``, ``..rpt-*``, ``..help-*``, состояния и визуальные подсказки.

optional

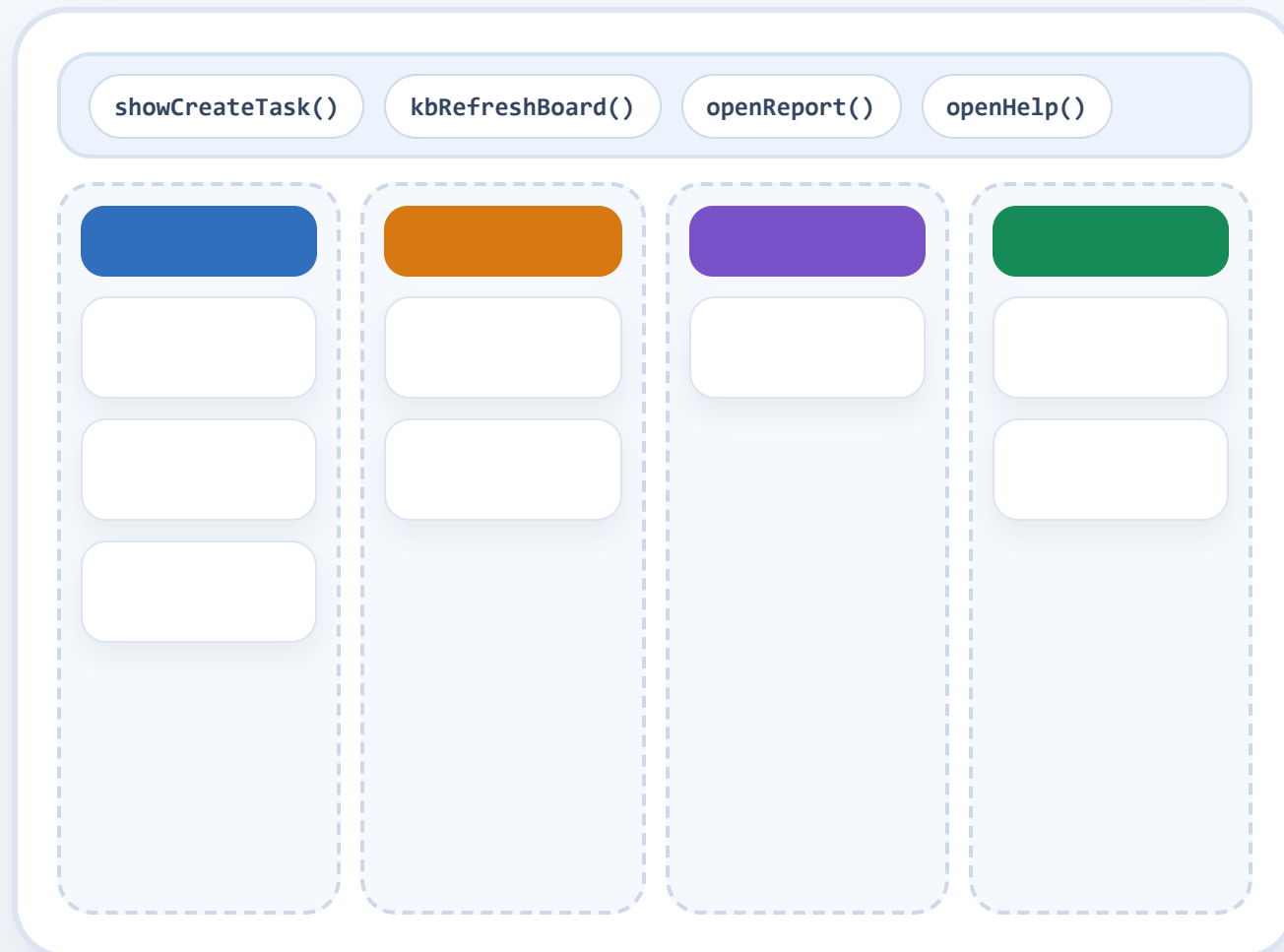
kanbanButton.cs

Команда открытия экрана ``UI\Screens\myKanbanTest``.

При открытии доски C# готовит данные, HTML только раскладывает их по колонкам.



HTML задаёт точки подключения: `id` ищет JS, `class` красит CSS, inline-события вызывают `window.\*`.



`id="kb-body-0..3"`

``kbDrop`` по родителю понимает старый статус карточки.

`id="kbc_{{t.id}}"`

JS находит конкретную карточку для удаления, бейджа, drag/drop.

`class="kb-card pri-..."`

CSS показывает приоритет и состояние карточки.

`id="tcmOverlay"`

Модалка управляется классом ``.visible`` из JS.

`id="reportBody"`

``renderReport`` заполняет таблицу отчёта.

Клиентский код закрыт в IIFE: наружу выставлены только функции, которые нужны HTML-обработчикам.

IIFE + public API

```
(function () {  
  var _kbH, _tcmData;  
  window.kbDrop = function() { ... };  
  window.tcmOpen = function() { ... };  
})();
```

1

`cal\*`

Календарь для панели создания и карточки задачи.

2

`kb\*`

Доска, drag/drop, создание, фильтры, иерархия.

3

`tcm\*`

Task Card Modal: открыть, отрисовать, сохранить, вложения, чат, история.

4

`rpt\*` / help

Отчёт и справка: overlay + render данных.

`window.external.InvokeTemplate()` — это местная “шина команд”, не AJAX и не REST.

kanban.js

Собирает строку параметров, валидирует UI, вызывает метод backend.

`InvokeTemplate("MethodName", "a|b|c")`

"MoveTask"

DoMoveTask()

"CreateTask"

DoCreateTask()

"GetTaskDetails"

DoGetTaskDetails()

"GetReport"

DoGetReport()

"AddComment"

DoAddComment()

C# Invoke

`switch(methodName)` выбирает `Do*`, работает с PLM и возвращает `OK`, JSON, ключ или `ERROR:*`.

Главный C#-файл — диспетчер команд. Каждая строка `case` — публичная операция для JS.

lifecycle

BeforeRender / RefreshBoard

Подготовить колонки, перерендерить экран.

col_0..3

renderArgs

tasks

CreateTask / MoveTask / SaveTask / DeleteTask

Создать, переместить, сохранить или удалить `KanbanTask`.

DoCreateTask

DoSaveTask

modal

GetTaskDetails / GetTaskHistory

Данные карточки, история изменений, права редактирования.

JSON

canFullEdit

hierarchy

GetHierarchyInfo / SetViewMode

Роль, подразделение, списки пользователей, текущий режим.

_kbH

KbViewMode

attachments

Pick* / Add* / Remove* / Open*

Объекты и контейнеры PLM, поиск, открытие, лимиты.

InfoObject

DataContainer

comments/report

AddComment / GetReport

Чат карточки и агрегированные показатели за период.

CommentsJSON

period|scope

Панель создания — самый показательный поток: HTML input → `doCreateTask` → C# create → optional attachments → refresh.

HTML

#kb-new-*

Поля: title, status, priority, dueDate, details, tags, assignee.

onclick="doCreateTask()"

JS

doCreateTask()

Тримит строки, проверяет дату, выбирает одиночный или групповой режим.

title|status|priority|dueDate|details|tags|assignee

C#

CreateTask / CreateGroupTask

Проверяет scope назначения, создаёт `KanbanTask`, заполняет атрибуты.

return nameKey или
ERROR:...

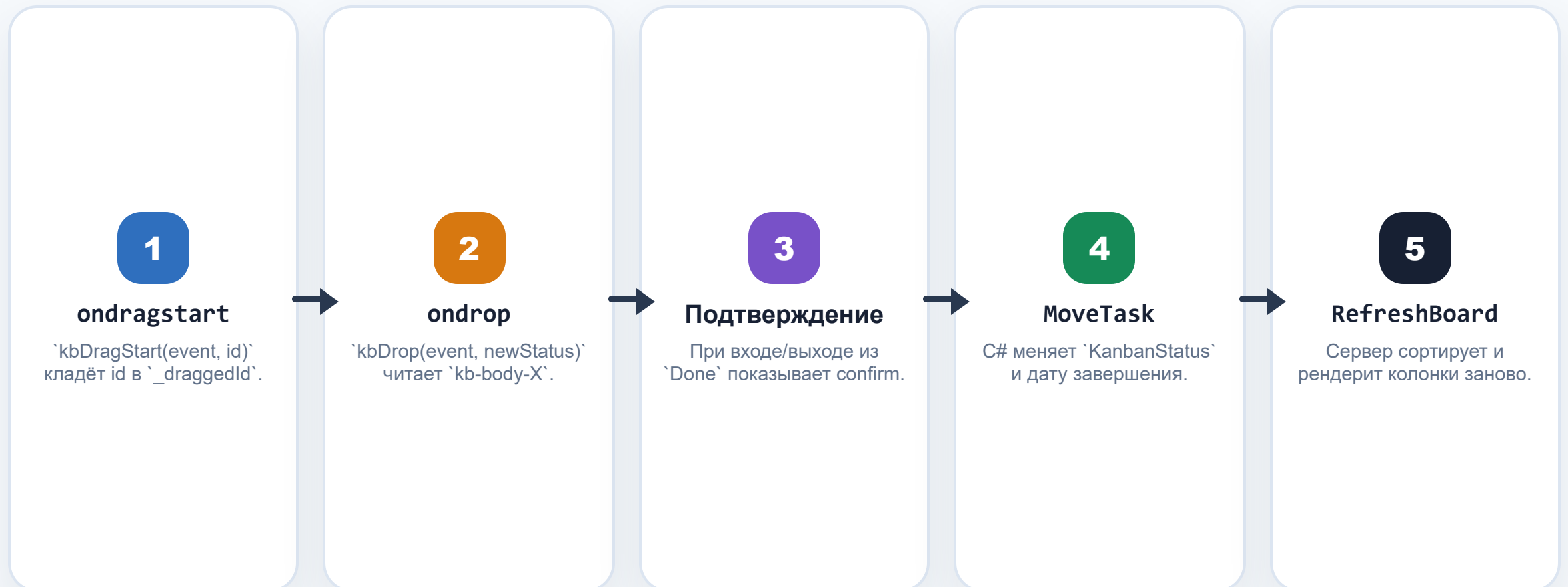
После

AddAttachment / RefreshBoard

Если были временные вложения, JS прикрепляет их к новой задаче и обновляет доску.

_crtPendingAttachments[]

Drag & drop не “двигает div навсегда”. Он вызывает ``MoveTask``, а потом сервер заново отдаёт правильный порядок.



Ключевой контракт: ``MoveTask`` получает строку ``taskId|newStatus``

Модалка — отдельный мини-модуль внутри `kanban.js`: загрузка JSON, render формы, сохранение, чат, вложения, история.

Карточка задачи

tcm

`kbOpenTask()`

Убирает бейдж «НОВАЯ», вызывает `tcmOpen(taskId)`.

`tcmOpen()`

``GetTaskDetails`` → JSON → ``_tcmData`` → `tcmRender()`.

`tcmRender()`

Заполняет поля, применяет ``canFullEdit``, рисует теги и метаданные.

`tcmSave()`

``SaveTask``:
``nameKey|title|status|prio|dueDate|tags|assignee|details``.

`tcmToggleRevs()`

История грузится лениво через ``GetTaskHistory`` только при раскрытии.

Фильтры — это не просто `select` на клиенте. Выбранный режим сохраняется на сервере и применяется при `BeforeRender`.

_kbH

```
role: "admin" | "headOfDept" | ...
myKey / myContext
viewMode: "my" | "sector" | ...
divisions[] / sectors[] / users[]
```

kbInitHierarchy()

``GetHierarchyInfo`` загружает роль, контекст и списки для UI.

kbApplyMode()

Собирает режим из `dept/sector/user select`.

SetViewMode

Пишет режим в ``obj.PropertyBag["KbViewMode"]``.

RefreshBoard

Перезапускает `render` с новым `scope`.

GetAllowedUserIdSet

Backend решает, какие Assignee можно показывать.

view modes

``my`, `all`, `dept`, `sector`, `user:KEY`, `group:CTX``.

У карточки есть два типа связей PLM и отдельный JSON-канал для комментариев.

objects

InfoObject

`PickObjects`, `PickAndAttach`,
`AddAttachment`, `RemoveAttachment`,
`OpenObject`, `SearchObjects`.

AttachedObjects

tcmAtt*

containers

DataContainer

`PickContainers`, `PickAndAttachContainer`,
`AddContainer`, `RemoveContainer`,
`OpenContainer`.

AttachedContainers

folder/project

chat

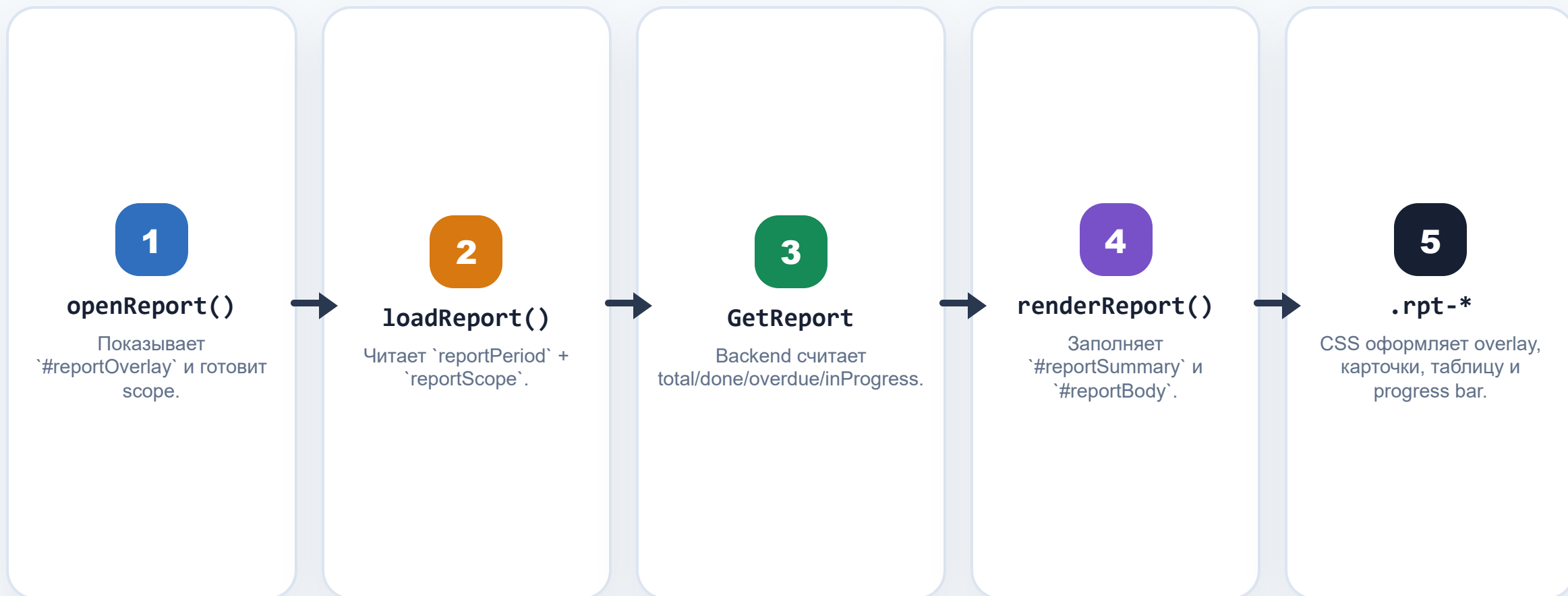
CommentsJSON

`GetComments`, `AddComment`,
`DeleteComment`; UI рисует список через
`tcmLoadComments`.

tcm-chat

ChangeLog

Отчёт — отдельный overlay: JS собирает период и score, backend считает задачи, JS рисует сводку и таблицу.



CSS разбит не по файлам, а по префиксам компонентов. JS часто не меняет style напрямую, а переключает классы-состояния.

■ **.kb-***

`.kb-board` `.kb-col` `.kb-card` `.kb-create-panel` `.kb-cal`
`.kb-tag`

■ **.tcm-***

`.tcm-overlay.visible` `.tcm-dialog` `.tcm-att-list` `.tcm-chat`
`.tcm-revs`

■ **state classes**

`.drag-over` `.kb-card-new` `.pri-urgent` `.kb-sel-urgent`
`.tcm-header-urgent`

■ **.rpt-* / .help-***

`.rpt-overlay` `.rpt-table` `.rpt-bar-fill` `.help-dialog`
`.help-table`

— Данные, которые связывают всё вместе

В UI карточка выглядит просто, но за ней стоит `KanbanTask` с атрибутами, справочниками и user-контекстом.

Container

All_Kanban_Tasks_Folder

template: All_Kanban_Tasks

`BeforeRender` перебирает `RootInfoObjects` этого контейнера.

KanbanTask fields

TaskName / TaskDetails

KanbanStatus / Priority

DueDate / CompletedDate

Assignee / Creator

ChangeLog / CommentsJSON

AttachedObjects / AttachedContainers

User + refs

Comission

Context

Ref_KanbanStatus

Ref_KanbanPriority

Ref_KanbanTags

Роли и фильтры завязаны на должность и подразделение.

Практическая навигация: начинай с симптома, потом открывай нужный слой.

Новая кнопка / поле

HTML id/class → window.function → CSS

Добавить DOM, экспортировать JS-функцию, при необходимости стилизовать.

Новое действие сервера

JS InvokeTemplate → C# case → Do*

Сразу зафиксировать формат payload и формат ответа.

Изменить карточку

BuildCardData → Liquid card → .kb-card

Данные карточки приходят из C#, внешний вид живёт в HTML/CSS.

Изменить модальку

GetTaskDetails ↔ tcmRender ↔ SaveTask

Поля должны совпадать в JSON, DOM id и save payload.

Изменить роли

GetHierarchyInfo + GetAllowedUserIdSet + _kbH

Фронт показывает варианты, backend принимает окончательное решение.

Только визуал

kanban.css: .kb-* / .tcm-* / .rpt-*

Если не меняешь DOM и payload, C# трогать не нужно.